

Advanced Scheduling Systems

Lecture Notes

Luca Simonetti

2025–2026

Contents

1	Introduction to Scheduling	3
1.1	Course Overview	3
1.2	Project Scheduling	3
1.2.1	Problem definition	4
1.2.2	What we leave out (for now)	4
1.3	The Critical Path Method (CPM)	5
1.3.1	CPM Forward (earliest schedule)	5
1.3.2	CPM Backward (latest schedule)	6
1.3.3	The critical path	6
1.3.4	Optimality of CPM	7
1.4	Greedy Algorithms	8
1.5	Algorithmic Analysis: Two Key Questions	8
1.6	Instance Format	9
1.7	The Knapsack Problem	9
1.7.1	Three greedy strategies	9
1.7.2	Upper bounds via relaxation	10
1.8	Easy vs Hard Problems	11
1.9	Resource-Constrained Project Scheduling (RCPSP)	12
1.9.1	Why RCPSP is hard	13
1.9.2	Lower bound for RCPSP	13
1.9.3	Greedy heuristics for RCPSP	13
1.9.4	RCPSP extensions	13
1.10	Shop Scheduling Problems	14
1.10.1	Job Shop Scheduling (JSS)	14
1.10.2	Flow Shop Scheduling (FSS)	15
1.10.3	Open Shop Scheduling (OSS)	15
1.11	Graph Colouring	16
1.11.1	Greedy colouring	16
1.11.2	DSatur	17
1.12	Timetabling Problems	18
1.12.1	Educational timetabling	18
1.12.2	Workforce timetabling: nurse rostering	19
1.12.3	Sport timetabling: round-robin tournaments	20
1.13	Routing Problems	22
1.13.1	The Travelling Salesman Problem (TSP)	22
1.13.2	Vehicle Routing Problem (VRP)	23
1.13.3	VRP variants	23
1.14	Bin Packing	24
1.14.1	1D Bin Packing	24
1.14.2	2D and 3D Bin Packing	26

2	C++ Implementation Framework	28
2.1	C++ Recap	28
2.1.1	Encapsulation and information hiding	28
2.1.2	Constructors	28
2.1.3	File I/O: <code>ifstream</code> and <code>ofstream</code>	29
2.1.4	The <code>vector</code> container	29
2.2	Software Architecture for Optimisation	29
2.3	Example: Graph Colouring I/O Classes	30
2.3.1	The <code>Graph</code> class (Input)	30
2.3.2	The <code>Coloring</code> class (Output)	31
2.4	Implementing Greedy: DSatur for Graph Colouring	32
2.4.1	The solver loop	32
2.4.2	Node selection (DSatur rule)	33
2.4.3	Propagation	33
2.4.4	Driver and Makefile	33
2.5	Example 2: Bus Driver Scheduling (I/O classes)	34
3	Single-Machine Scheduling	36
4	Parallel-Machine Scheduling	37
5	Flow Shop & Job Shop Scheduling	38
6	Metaheuristics for Scheduling	39

Chapter 1

Introduction to Scheduling

1.1 Course Overview

This course studies *discrete optimisation problems* and, crucially, how to *implement* algorithms that solve them. Think of it as *applied operations research*: we take the mathematical models you have seen in Operations Research and turn them into running code.

Discrete = integer-valued decisions: pick item 1, 2, 3... not a continuous value like 0.37.

The programme breaks into four blocks:

1. **Problem survey** (Lectures 1–4). A tour of scheduling, timetabling, and logistics problems, together with the first solving technique: greedy algorithms.
2. **Constraint modelling with MiniZinc**. We write a mathematical specification of the problem in the MiniZinc language; an underlying constraint-programming solver does the rest.
3. **Advanced C++ and frameworks**. Inheritance, virtual functions, abstract classes, and the object-oriented framework we use for local search.
4. **Solving techniques** (implemented in C++). Greedy, enumeration/backtracking, constraint programming (via MiniZinc), local search, and meta-heuristics. Each technique is studied briefly in theory, then immediately applied to a concrete problem.

Remark 1.1.1. The exam is a **project**: choose a problem (proposed by the instructor or negotiated), solve it with at least two different techniques, and compare the results. Groups of at most two people; individual contributions are assessed during an oral discussion.

1.2 Project Scheduling

Every project—building a house, developing a product, installing a software stack—consists of activities with dependencies between them. The *project scheduling problem* asks: given the activities and their dependencies, when should each activity start so that the whole project finishes as early as possible?

apt install internally solves a project-scheduling instance over package dependencies.

1.2.1 Problem definition

Definition 1.2.1 (Project Scheduling Problem). We are given:

- a set of n jobs (activities) $\{1, \dots, n\}$, each with integer duration d_i ;
- a set of *precedence constraints* $P \subseteq \{1, \dots, n\}^2$, forming a **DAG** (directed acyclic graph): $(i, j) \in P$ means job j cannot start before job i finishes.

We must assign a *start time* $S_i \geq 0$ to every job so that:

1. all precedences are respected: $S_j \geq S_i + d_i$ for every $(i, j) \in P$;
2. the *makespan* $C_{\max} = \max_i (S_i + d_i)$ is minimised.

The completion time of job i is $C_i = S_i + d_i$.

Let's unpack this. We have a collection of jobs, each taking a known amount of time. Some jobs depend on others: job j can only begin once every predecessor has finished. The dependencies form a directed acyclic graph—a DAG—because a cycle would make the problem infeasible (if A must precede B and B must precede A , nothing can ever start). Our goal is a single number: the makespan $C_{\max}$, i.e. the moment the *last* job finishes. We want it as small as possible.

Example 1.2.2 (Seven-job instance). Consider seven jobs with the following durations and precedence graph:

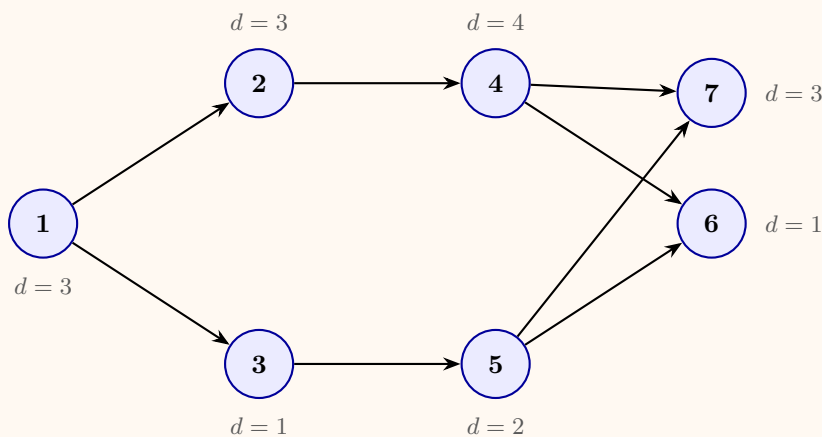


Figure 1.1: Precedence DAG for the seven-job instance (theorem 1.2.2). Each node shows the job index; durations are labelled alongside.

Reading the graph: job 1 must finish before jobs 2 and 3 can start; job 4 requires job 2; job 5 requires job 3; both jobs 6 and 7 require jobs 4 and 5.

1.2.2 What we leave out (for now)

The formulation above is deliberately simple. Real-world projects involve several complications that we set aside here but will revisit in later lectures:

- **Resources.** Two independent jobs may still conflict if they both need the same machine or worker. Resource constraints turn the problem from polynomial to NP-hard.
- **Release dates and deadlines.** A job may not be available before a certain date, or may have a hard deadline.

- **Uncertain durations.** In practice durations are stochastic (it might rain, a delivery might be late). One then minimises expected or worst-case makespan—this belongs to the Project Management course.
- **Generalised precedences.** Start-to-start, finish-to-finish, or time-lag constraints (e.g. concrete must be poured within a window after mixing).
- **Variable durations with costs.** Crashing: pay more to shorten a job. This introduces a cost–time trade-off (multi-objective).

1.3 The Critical Path Method (CPM)

The Critical Path Method is a classical greedy algorithm that solves the basic project scheduling problem (theorem 1.2.1) *optimally*.

The idea is disarmingly simple: schedule every job as early as possible, respecting precedences. We formalise this in two passes—forward and backward—and then combine them to identify the *critical path*.

*Walker & Kelley, late 1950s.
Originally for plant
maintenance shutdowns at
DuPont.*

1.3.1 CPM Forward (earliest schedule)

Definition 1.3.1 (CPM Forward). Process jobs in *topological order* of the precedence DAG:

1. For every job i with no predecessors, set $S_i = 0$.
2. For every other job i (once all predecessors are scheduled):

$$S_i = \max_{(k,i) \in P} C_k = \max_{(k,i) \in P} (S_k + d_k).$$

3. Set $C_i = S_i + d_i$.

The makespan is $C_{\max} = \max_{i=1}^n C_i$.

In words: each job starts at the earliest moment all its predecessors have finished. We simply walk through the DAG from sources to sinks, computing start and finish times as we go.

Example 1.3.2 (CPM Forward on the seven-job instance). Applying CPM Forward to theorem 1.2.2:

Job	1	2	3	4	5	6	7
d_i	3	3	1	4	2	1	3
S_i	0	3	3	6	4	10	10
C_i	3	6	4	10	6	11	13

Figure 1.2: Earliest start and completion times (CPM Forward). Highlighted cells belong to the critical path.

Let's trace the computation step by step:

- Job 1 has no predecessors: $S_1 = 0$, $C_1 = 3$.
- Jobs 2, 3 depend only on job 1: $S_2 = C_1 = 3$, $C_2 = 6$; $S_3 = C_1 = 3$, $C_3 = 4$.
- Job 4 depends on job 2: $S_4 = C_2 = 6$, $C_4 = 10$.

- Job 5 depends on job 3: $S_5 = C_3 = 4$, $C_5 = 6$.
 - Job 6 depends on jobs 4 and 5: $S_6 = \max(C_4, C_5) = \max(10, 6) = 10$, $C_6 = 11$.
 - Job 7 depends on jobs 4 and 5: $S_7 = \max(C_4, C_5) = 10$, $C_7 = 13$.
- The makespan is $C_{\max} = \max(C_6, C_7) = 13$.

1.3.2 CPM Backward (latest schedule)

The backward pass assumes we already know the makespan M from the forward pass and asks: what is the *latest* each job can start without increasing M ?

Definition 1.3.3 (CPM Backward). Let $M = C_{\max}$ from the forward pass. Process jobs in *reverse* topological order:

1. For every job i with no successors, set $C_i^{\text{late}} = M$.
2. For every other job i (once all successors are scheduled):

$$C_i^{\text{late}} = \min_{(i,j) \in P} S_j^{\text{late}}.$$

3. Set $S_i^{\text{late}} = C_i^{\text{late}} - d_i$.

Notice the symmetry: forward uses max over predecessors; backward uses min over successors. The forward pass pushes everything as early as possible; the backward pass pulls everything as late as possible.

Example 1.3.4 (CPM Backward on the seven-job instance). With $M = 13$ from theorem 1.3.2:

Job	1	2	3	4	5	6	7
S_i^{late}	0	3	7	6	8	12	10
C_i^{late}	3	6	8	10	10	13	13

Figure 1.3: Latest start and completion times (CPM Backward).

Tracing backwards from the sinks:

- Jobs 6 and 7 have no successors: $C_6^{\text{late}} = C_7^{\text{late}} = 13$. So $S_6^{\text{late}} = 12$, $S_7^{\text{late}} = 10$.
- Job 4: successors are 6 and 7. $C_4^{\text{late}} = \min(S_6^{\text{late}}, S_7^{\text{late}}) = \min(12, 10) = 10$, $S_4^{\text{late}} = 6$.
- Job 5: successors are 6 and 7. $C_5^{\text{late}} = \min(12, 10) = 10$, $S_5^{\text{late}} = 8$.
- Job 2: successor is 4. $C_2^{\text{late}} = S_4^{\text{late}} = 6$, $S_2^{\text{late}} = 3$.
- Job 3: successor is 5. $C_3^{\text{late}} = S_5^{\text{late}} = 8$, $S_3^{\text{late}} = 7$.
- Job 1: successors are 2 and 3. $C_1^{\text{late}} = \min(3, 7) = 3$, $S_1^{\text{late}} = 0$.

1.3.3 The critical path

Definition 1.3.5 (Critical activity and critical path). A job i is *critical* if its earliest and latest start times coincide: $S_i = S_i^{\text{late}}$. The *critical path* is the set of all critical activities (and the precedence arcs connecting them).

A critical job has *zero slack*: any delay in its execution directly increases the makespan. Non-critical jobs have *float*—they can shift within the interval $[S_i, S_i^{\text{late}}]$ without affecting $C_{\max}$.

Example 1.3.6 (Critical path of the seven-job instance). Comparing the forward and backward tables:

Job	1	2	3	4	5	6	7
S_i (earliest)	0	3	3	6	4	10	10
S_i^{late} (latest)	0	3	7	6	8	12	10
Slack	0	0	4	0	4	2	0
Critical?	✓	✓		✓			✓

The critical path is $1 \rightarrow 2 \rightarrow 4 \rightarrow 7$, with a total duration of $3 + 3 + 4 + 3 = 13 = C_{\max}$.

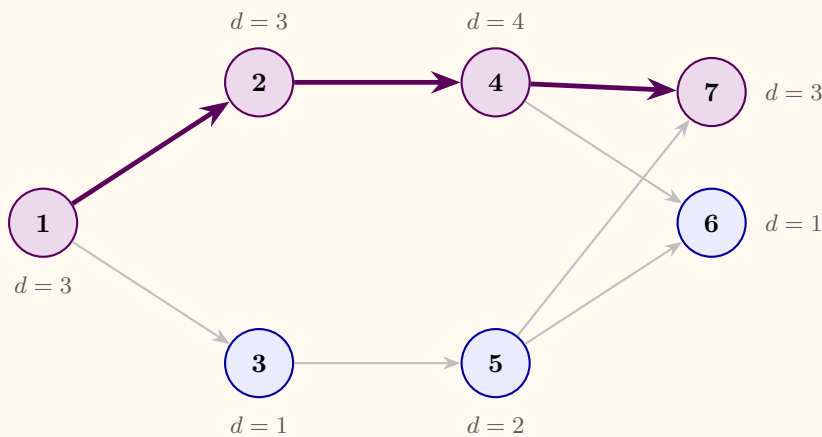


Figure 1.4: The critical path $1 \rightarrow 2 \rightarrow 4 \rightarrow 7$ (violet nodes and edges). Non-critical jobs and arcs are shown in gray.

Job 3, for instance, can start anywhere between time 3 (earliest) and time 7 (latest) without affecting the makespan. Job 7, however, is locked: if it starts even one unit later, the project finishes later.

1.3.4 Optimality of CPM

Theorem 1.3.7 (CPM is optimal). *The CPM Forward schedule achieves the minimum possible makespan for the project scheduling problem (theorem 1.2.1).*

■ Formal details — Optimality of CPM

The proof is by contradiction. Consider the critical path $i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_k$ found by CPM. Every job on this path starts exactly when its predecessor finishes (zero slack), so the makespan equals

$$C_{\max} = d_{i_1} + d_{i_2} + \dots + d_{i_k}.$$

Any feasible schedule must respect the precedence chain $i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_k$, which means

$$C_{\max}^* \geq d_{i_1} + d_{i_2} + \dots + d_{i_k} = C_{\max}.$$

Since CPM produces a feasible schedule with makespan $C_{\max}$, and no feasible schedule can do better, CPM is optimal. $\square$

The key insight is that the critical path acts as a *lower bound*: no matter how cleverly we rearrange jobs, we can never avoid executing every job on the critical path in sequence, so the makespan is at least the sum of their durations. CPM *achieves* this bound, so it is optimal.

1.4 Greedy Algorithms

CPM is our first example of a **greedy algorithm**.

Italian: algoritmo avido; *the English* “greedy” is standard.

Definition 1.4.1 (Greedy algorithm (informal)). A greedy algorithm builds a solution incrementally. At each step it makes the locally best choice (assign the earliest possible start time, pick the shortest job, etc.) and *never reconsiders* that choice.

Greedy algorithms are typically very fast—often polynomial time—because they never backtrack. The catch is that for many problems the locally best choice is not globally optimal. CPM is a happy exception: the greedy “start as early as possible” rule happens to be provably optimal for the basic project scheduling problem.

Observation 1.4.2. For more complex scheduling problems (with resources, deadlines, or variable durations), greedy algorithms generally do *not* find the optimal solution. We will need more powerful techniques: enumeration, constraint programming, and local search.

1.5 Algorithmic Analysis: Two Key Questions

Whenever we study a new algorithm, we ask two questions:

1. **Computational complexity.** How does the running time scale with the input size? CPM processes each job once and inspects each precedence arc once, so its complexity is $\mathcal{O}(n + |P|)$ —linear in the size of the input.
2. **Exactness.** Does the algorithm always find the optimal solution? As we proved in theorem 1.3.7, CPM is *exact* for the basic project scheduling problem.

$|P|$ = number of arcs. Worst case $\mathcal{O}(n^2)$; in practice graphs are sparse, so nearly linear.

■ Intermezzo — CPM at scale

In the lecture demo, a C++ implementation of CPM solved an instance with **one million jobs** in about 4 seconds, and instances with over 10 million jobs in under a minute. The algorithm’s simplicity—a single topological-order pass through the DAG—translates directly into practical speed. This is the power of a well-chosen greedy algorithm on a problem where greedy happens

to be optimal.

1.6 Instance Format

For the project scheduling problem we use a simple text format:

Richer formats for more complex problems; this suffices for the basic case.

```

1 n (* number of jobs *)
2 d_1 #pred_1 pred_1a pred_1b ...
3 d_2 #pred_2 pred_2a pred_2b ...
4 ...
5 d_n #pred_n pred_na pred_nb ...

```

Listing 1.1: Instance file format for project scheduling.

The first line gives the number of jobs n . Each subsequent line i (for $i = 1, \dots, n$) contains: the duration d_i , the number of predecessors, and the list of predecessor indices.

Example 1.6.1 (Instance file for theorem 1.2.2).

```

7
3 0
3 1 1
1 1 1
4 1 2
2 1 3
1 2 4 5
3 2 4 5

```

Job 1 (line 2) has duration 3 and no predecessors. Job 6 (line 7) has duration 1 and two predecessors: jobs 4 and 5.

1.7 The Knapsack Problem

Before continuing with scheduling problems, we take a detour through a simpler combinatorial problem that lets us sharpen three important ideas: greedy heuristics, upper bounds, and the distinction between easy and hard problems.

Not a scheduling problem, but the perfect sandbox for greedy, bounds, and complexity concepts.

Definition 1.7.1 (0/1 Knapsack Problem). We are given n items, each with a *weight* w_i and a *value* v_i , and a knapsack of capacity W . We must choose a subset of items that fits in the knapsack and maximises total value:

$$\max \sum_{i=1}^n v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

In plain words: every item is either taken ($x_i = 1$) or left behind ($x_i = 0$). We want as much value as possible without exceeding the weight limit. Think of packing a backpack for a hike, or—in computing—selecting files to store on a disk with limited capacity.

1.7.1 Three greedy strategies

Example 1.7.2 (Six-item knapsack instance). Consider six items with capacity $W = 100$:

Item	1	2	3	4	5	6
Weight w_i	100	50	45	20	10	5
Value v_i	40	35	18	4	10	2
Ratio v_i/w_i	0.40	0.70	0.40	0.20	1.00	0.40

Figure 1.5: Data for the six-item knapsack instance (theorem 1.7.2).

Let's try three natural greedy rules on this instance:

Greedy 1: highest value first. Pick the item with the largest v_i that still fits.

- Select item 1 ($v = 40$, $w = 100$). Remaining capacity: 0.
- No more items fit.
- **Total value: 40.**

Greedy 2: lightest first. Pick the item with the smallest w_i that still fits.

- Select item 6 ($w = 5$), then 5 ($w = 10$), then 4 ($w = 20$), then 3 ($w = 45$). Remaining capacity: $100 - 80 = 20$.
- Items 1 and 2 do not fit.
- **Total value:** $2 + 10 + 4 + 18 = 34$.

Greedy 3: best value-to-weight ratio. Pick the item with the highest v_i/w_i that still fits.

- Select item 5 (ratio 1.00, $w = 10$). Remaining: 90.
- Select item 2 (ratio 0.70, $w = 50$). Remaining: 40.
- Tied at ratio 0.40: items 1, 3, 6. Item 1 ($w = 100$) does not fit; item 3 ($w = 45$) does not fit; item 6 ($w = 5$) fits. Remaining: 35.
- Item 4 (ratio 0.20, $w = 20$) fits. Remaining: 15.
- **Total value:** $10 + 35 + 2 + 4 = 51$. Weight used: 85.

*The classic knapsack heuristic.
Ties broken by largest item first.*

None of these finds the optimal solution. By inspection, items $\{2, 3, 6\}$ weigh exactly $50 + 45 + 5 = 100$ and achieve value $35 + 18 + 2 = \mathbf{55}$. The best greedy (ratio-based) left 15 units of capacity unused, missing a better packing.

1.7.2 Upper bounds via relaxation

We found a solution worth 55 by inspection, but how do we *prove* no solution can do better? The standard technique is *relaxation*: solve an easier version of the problem whose optimal value is guaranteed to be at least as good.

For a maximisation problem, a relaxation gives an upper bound. For minimisation (like makespan), it gives a lower bound.

Definition 1.7.3 (Fractional Knapsack). The same formulation as theorem 1.7.1, but with $x_i \in [0, 1]$ instead of $x_i \in \{0, 1\}$. Items may be taken in fractions: if only half of item i fits, we take half and get $\frac{1}{2}v_i$.

Since any 0/1 solution is also a valid fractional solution, the fractional optimum is an upper bound on the integer optimum. And the fractional problem has a simple exact greedy:

Observation 1.7.4. The greedy algorithm that sorts items by decreasing v_i/w_i and takes as much of each item as fits is *optimal* for the fractional knapsack.

Example 1.7.5 (Fractional bound for theorem 1.7.2). Sort by ratio: item 5 (1.00), item 2 (0.70), then the 0.40-ratio group.

- Take all of item 5: weight 10, value 10. Remaining: 90.
- Take all of item 2: weight 50, value 35. Remaining: 40.
- Item 1 weighs 100; take $40/100 = 0.4$ of it: weight 40, value $0.4 \times 40 = 16$. Remaining: 0.

Total value: $10 + 35 + 16 = 61$. Since the integer optimum ≤ 61 and we found a feasible integer solution worth 55, the optimality gap is at most $61 - 55 = 6$.

Remark 1.7.6. The fractional bound does not prove that 55 is optimal (there could be an integer solution worth 56, 57, ..., 61). For this small instance we verified optimality by inspection. For large instances, tighter bounds or exact algorithms (branch-and-bound, dynamic programming) are needed.

1.8 Easy vs Hard Problems

So far we have seen two *easy* problems—project scheduling and fractional knapsack—where a greedy algorithm finds the optimum in polynomial time. The 0/1 knapsack, by contrast, is *hard*: no known polynomial-time algorithm solves it exactly.

Definition 1.8.1 (NP-hard (informal)). A problem is NP-hard if the best known exact algorithms require time exponential in the input size.

The key structural difference: for easy problems, the algorithm makes locally optimal choices with no branching. For hard problems, at each step there are multiple choices (which item to take, which job to schedule next), and exploring all combinations leads to an exponential number of candidate solutions—roughly 2^n for n binary decisions.

Whether NP-hard problems truly require exponential time is the open $P \stackrel{?}{=} NP$ question.

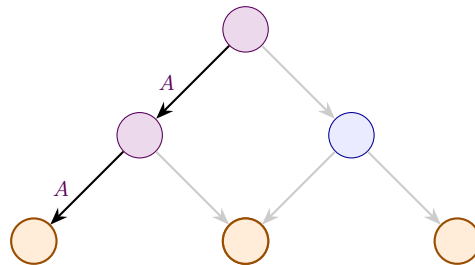


Figure 1.6: A greedy algorithm (violet path) picks one branch at each decision point, visiting n nodes total. An exact algorithm may need to explore all 2^n leaves.

A **greedy** algorithm traverses a single root-to-leaf path through this tree: fast (n steps), but no guarantee of finding the best leaf. An **exact** algorithm explores enough of the tree to certify optimality—potentially all of it.

Observation 1.8.2. Algorithms fall into two families:

- **Exact** algorithms guarantee the optimal solution, but may take exponential time on hard problems.
- **Heuristic** algorithms (greedy being the simplest subfamily) find good solutions quickly, without optimality guarantees.

This course covers both: we will implement greedy heuristics, enumeration (backtracking), constraint programming (via MiniZinc), and local search / metaheuristics.

1.9 Resource-Constrained Project Scheduling (RCPSP)

We now return to project scheduling and add the complication we set aside earlier: **resources**.

Definition 1.9.1 (Resource-Constrained Project Scheduling Problem—RCPSP). We are given the same data as the basic project scheduling problem (theorem 1.2.1), plus:

- R types of *renewable resources*, each with a capacity Q_r ($r = 1, \dots, R$);
- for each job i and resource r , the *resource demand* q_{ir} : how many units of resource r job i uses while executing.

Resources are *renewable*: a unit is occupied while a job runs, then becomes available again.

A schedule must satisfy:

1. all precedence constraints (as before);
2. at every time instant t , the total demand for each resource r from all jobs running at time t does not exceed Q_r .

The objective is to minimise the makespan $C_{\max}$.

Workers, machines: busy during the job, free afterwards.

Non-renewable resources are consumed permanently.

The resource constraints are what make RCPSP fundamentally harder than basic project scheduling. Without resources, CPM schedules every job as early as possible—there is no conflict. With resources, two jobs may both be ready to start, but starting them simultaneously would exceed a resource capacity. We must choose which one goes first, and that choice affects all subsequent decisions.

Example 1.9.2 (RCPSP with two resources). Take the seven-job instance from theorem 1.2.2 and add two resources with capacities $Q_1 = 4$, $Q_2 = 3$:

Job	1	2	3	4	5	6	7
Duration d_i	3	3	1	4	2	1	3
$q_{i,1}$ (R1)	3	2	3	1	0	2	1
$q_{i,2}$ (R2)	0	2	1	2	2	1	2

Figure 1.7: Resource demands for the seven-job RCPSP instance.

With CPM (ignoring resources), jobs 2 and 3 start simultaneously at time 3. But job 2 needs 2 units of R1 and job 3 needs 3: together that is $2 + 3 = 5 > 4 = Q_1$. They cannot run in parallel—we must choose. This is a **choice point** that does not exist in the unconstrained version.

1.9.1 Why RCPSP is hard

Each choice point roughly doubles the number of candidate schedules. With n jobs, the search space grows to approximately 2^n alternatives. For $n = 100$, that is $2^{100} \approx 10^{30}$ schedules—impossible to enumerate.

2^n is a rough upper bound; the true number depends on the graph and resource constraints.

RCPSP is NP-hard. The PSPLib benchmark library contains instances with 30, 60, 90, and 120 jobs; some 120-job instances remain *unsolved to optimality* to this day.

1.9.2 Lower bound for RCPSP

A natural lower bound: **ignore the resource constraints** and solve the resulting basic project scheduling problem with CPM. Since removing constraints can only improve (or maintain) the optimal value:

$$C_{\max}^{\text{CPM}} \leq C_{\max}^{\text{RCPSP*}}.$$

If a heuristic solution happens to match this bound, we know it is optimal. Otherwise, the gap $C_{\max}^{\text{heuristic}} - C_{\max}^{\text{CPM}}$ tells us how far we *might* be from optimality (though the true optimum could lie anywhere in between).

1.9.3 Greedy heuristics for RCPSP

A greedy heuristic for RCPSP extends the CPM idea: schedule one job at a time, choosing the *earliest feasible* start that respects both precedences and resources. The question is: **which job to schedule next?**

There are two main families of greedy rules:

Time-based (schedule generation scheme). Advance time instant by instant. At each moment, identify all jobs that *could* start (predecessors done, resources available) and pick one by a priority rule: longest processing time, shortest processing time, most resource demand, etc.

Job-based (priority rule). Maintain three sets:

- *Scheduled*: jobs already placed.
- *Ready*: jobs whose predecessors are all scheduled.
- *Not ready*: at least one predecessor is unscheduled.

At each step, select a job from the *ready* set by a priority rule, then find the earliest time at which it can start without violating resource capacities.

Remark 1.9.3. Because the greedy traverses a single path through the exponential search tree, different runs with different priority rules—or even random tie-breaking—can produce very different makespans. In the seven-job example, random restarts yielded solutions ranging from $C_{\max} = 14$ to $C_{\max} = 16$, against a CPM lower bound of 12.

1.9.4 RCPSP extensions

Several extensions make the problem richer (and harder):

- **Multi-mode RCPSP.** Each job has several *execution modes*, each with its own duration and resource demands. For instance, digging a trench might take 3 days with 3 workers, or 2 days with 1 worker and an

excavator. The decision now includes both *when* and *how* to execute each job.

- **Non-renewable resources.** Resources that are *consumed* (concrete, fuel, budget). In the single-mode case, total consumption is fixed regardless of schedule—feasibility reduces to checking whether the total demand exceeds supply. Combined with multiple modes, however, the choice of mode affects consumption, making the problem significantly harder.
- **Preemption.** A job may be *interrupted* and resumed later, freeing its resources for another job in the meantime. Whether preemption is realistic depends on the application: pausing a software build is easy; pausing a concrete pour is not.
- **Stochastic durations.** Durations drawn from probability distributions. Objectives become expected or worst-case makespan—this variant belongs more to the Project Management course.

1.10 Shop Scheduling Problems

We now introduce a family of scheduling problems where *machines* (processors) are the central resource. These are called **shop scheduling** problems.

“Shop” = *workshop/factory floor, not a retail store.*

The common setup: we have n jobs and m machines. Each job consists of a sequence of *tasks* (also called *operations*), where each task must be processed on a specific machine for a given duration. A machine can process at most one task at a time, and (unless stated otherwise) a task cannot be interrupted once started.

1.10.1 Job Shop Scheduling (JSS)

Definition 1.10.1 (Job Shop Scheduling Problem). We are given:

- n jobs, each consisting of an *ordered* sequence of tasks;
- m machines;
- each task T_{ij} (the j -th task of job i) is assigned to a specific machine μ_{ij} and has duration d_{ij} ;
- (optionally) a *release date* r_i and a *due date* $\bar{d}_i$ for each job.

Constraints:

1. *Precedence within each job:* the tasks of a job must be processed in their given order.
2. *Machine capacity:* each machine processes at most one task at a time.
3. Release dates are hard constraints.

Objective: minimise the makespan $C_{\max}$, or minimise total *tardiness* $\sum_i T_i$, where $T_i = \max(0, C_i - \bar{d}_i)$ is the delay of job i past its due date.

Example 1.10.2 (A small job shop). Three jobs on three machines:

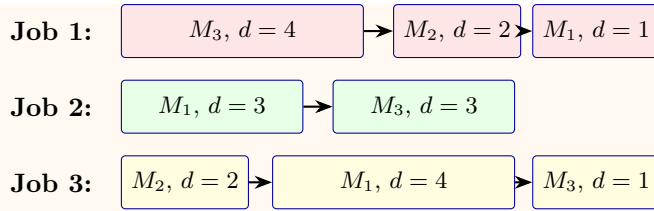


Figure 1.8: Three jobs in a job shop. Each row shows one job's task sequence with its assigned machine and duration. Arrows indicate precedence within the job.

Job 1 visits machines $3 \rightarrow 2 \rightarrow 1$; job 2 visits $1 \rightarrow 3$; job 3 visits $2 \rightarrow 1 \rightarrow 3$. The scheduler must decide, for each machine, in what order to process the tasks assigned to it—without violating the within-job precedences.

1.10.2 Flow Shop Scheduling (FSS)

Definition 1.10.3 (Flow Shop Scheduling Problem). A special case of the job shop where **all jobs visit the machines in the same fixed order** $M_1 \rightarrow M_2 \rightarrow \dots \rightarrow M_m$. Every job has exactly m tasks (some may have duration zero).

The crucial simplification: since the machine order is identical for every job, the *only* decision is the **permutation** (sequence) in which jobs enter the shop. Once the permutation is fixed, start times follow deterministically: each task starts as soon as both its machine is free and the previous task of the same job is finished.

If all machines use the same permutation, we have a permutation flow shop—the most common variant.

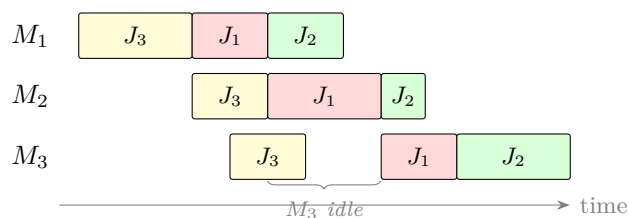


Figure 1.9: A flow shop Gantt chart for the permutation J_3, J_1, J_2 . Machine M_3 sits idle between J_3 and J_1 because it must wait for J_1 to finish on M_2 . A different permutation might reduce this idle time.

The goal is to find the permutation that minimises idle gaps and thus the makespan. Despite this structural simplification, the flow shop with $m \geq 3$ machines is NP-hard.

1.10.3 Open Shop Scheduling (OSS)

Definition 1.10.4 (Open Shop Scheduling Problem). Like the job shop, but with **no fixed ordering** of tasks within a job. Each job still has one task per machine, but the scheduler is free to choose in which order a job visits the machines.

In the open shop, the within-job precedences disappear entirely. What ties a job's tasks together are its release date and due date: all tasks of a job must

start after its release and finish before its deadline. The scheduler must decide both the *order of tasks within each job* and the *order of jobs on each machine*.

	Machine order	Decision variable	Complexity
Job shop	job-specific, fixed	start time of each task	NP-hard
Flow shop	same for all jobs	job permutation	NP-hard ($m \geq 3$)
Open shop	free	task order + start times	NP-hard ($m \geq 3$)

■ Intermezzo — Make as a project scheduler

When you type `make` to compile a C++ project, the build tool reads the dependency graph from the Makefile and solves a project scheduling problem: each compilation command is a job, its duration is the compilation time, and “`A.o` depends on `A.cc`” is a precedence constraint.

With `make -j` (parallel mode), Make schedules independent compilations on separate CPU cores—exactly the CPM forward strategy. In a classroom demo, a synthetic Makefile encoding the seven-job instance from theorem 1.2.2 (using `sleep` to simulate durations) completed in 12 seconds with `make -j`, matching the CPM makespan, versus 16 seconds in sequential mode.

1.11 Graph Colouring

Many scheduling and timetabling problems reduce, at their core, to assigning *labels* (colours) to objects so that conflicting objects receive different labels. This is precisely the **graph colouring** problem.

Applications: register allocation, frequency assignment, map colouring.

Definition 1.11.1 (Graph Colouring—two formulations). Given an undirected graph $G = (V, E)$:

1. **Search version.** Given an integer k , find a *proper k -colouring*: an assignment $c: V \rightarrow \{1, \dots, k\}$ such that $c(u) \neq c(v)$ for every edge $\{u, v\} \in E$.
2. **Optimisation version.** Find the minimum k for which a proper k -colouring exists. This minimum is the *chromatic number* $\chi(G)$.

The search version asks: “can I do it with k colours?” The optimisation version asks: “what is the fewest colours I need?” Both are NP-hard in general (for $k \geq 3$).

1.11.1 Greedy colouring

The simplest approach: process vertices one by one and assign each the *smallest colour not used by its already-coloured neighbours*. The quality depends entirely on the **vertex ordering**.

Maximum-degree ordering. Sort vertices by decreasing degree (number of neighbours) and colour in that order. The intuition: high-degree vertices are the most constrained, so handle them first while many colours are still available.

Example 1.11.2 (Greedy colouring by max degree). Consider a graph on five vertices:

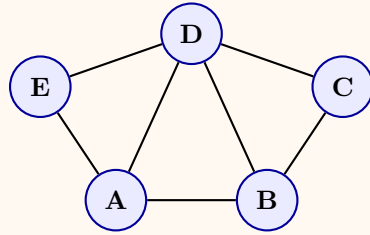


Figure 1.10: A small graph. Vertex D has the highest degree (4), followed by A and B (degree 3 each).

Degrees: $D = 4$, $A = 3$, $B = 3$, $C = 2$, $E = 2$. Processing in order D, A, B, C, E :

1. D : no coloured neighbours $\Rightarrow$ colour 1.
2. A : neighbour D has colour 1 $\Rightarrow$ colour 2.
3. B : neighbours $D(1)$, $A(2)$ $\Rightarrow$ colour 3.
4. C : neighbours $B(3)$, $D(1)$ $\Rightarrow$ colour 2.
5. E : neighbours $A(2)$, $D(1)$ $\Rightarrow$ colour 3.

Result: 3 colours. In fact $\chi(G) = 3$ here, so the greedy found the optimum—but this is not guaranteed in general.

1.11.2 DSatur

A smarter ordering is **DSatur** (Degree of Saturation), which chooses the next vertex *dynamically* rather than fixing the order upfront.

Brélaz, 1979. Still competitive with far more complex heuristics.

Definition 1.11.3 (Saturation degree). The *saturation degree* of a vertex v is the number of *distinct colours* already assigned to its neighbours.

Definition 1.11.4 (DSatur algorithm). Repeat until all vertices are coloured:

1. Among uncoloured vertices, pick the one with the **highest saturation degree**. Break ties by highest (uncoloured) degree; break further ties arbitrarily.
2. Assign it the smallest colour not used by its neighbours.

The intuition: a vertex with high saturation is surrounded by many different colours, so it is the most “urgent” to colour next—it has the fewest options left. By always attending to the most constrained vertex, DSatur avoids wasteful colour introductions more often than a static ordering.

Example 1.11.5 (DSatur on the same graph). Using the graph from fig. 1.10:

1. All saturations are 0; pick D (highest degree 4) $\Rightarrow$ colour 1.
2. Saturations: $A = 1$, $B = 1$, $C = 1$, $E = 1$. Tie; pick A (degree 3) $\Rightarrow$ colour 2.
3. Saturations: $B = 2$, $C = 1$, $E = 2$. Tie between B and E ; pick B (degree 3) $\Rightarrow$ colour 3.

4. Saturations: $C = 2$, $E = 2$. Tie; pick C (degree 2) $\Rightarrow$ colour 2.
5. E : saturation 2 $\Rightarrow$ colour 3.

Same result (3 colours), but on larger graphs DSatur typically outperforms static max-degree ordering.

Remark 1.11.6. DSatur is exact for bipartite graphs (always finds $\chi = 2$) and for cycle graphs. For general graphs it is a heuristic: it may use more than $\chi(G)$ colours.

1.12 Timetabling Problems

Timetabling is the task of assigning events (exams, lectures, shifts, matches) to time slots and resources (rooms, people, fields) subject to a rich set of constraints.

We survey three major families: educational, workforce, and sport timetabling.

At their core, timetabling = graph colouring with richer constraints.

1.12.1 Educational timetabling

Examination timetabling. Assign exams to time slots so that no student has two exams at the same time. The conflict graph has one vertex per exam and an edge whenever two exams share at least one enrolled student. The problem then reduces to graph colouring (minimise the number of time slots, or fit within a given number). Additional soft constraints typically include:

- spreading each student's exams over the available days (avoid two exams on the same day or on consecutive days);
- room capacity limits;
- preferred time slots for large exams.

Course timetabling. Assign lectures to weekly time slots and rooms. The *curriculum-based* variant defines conflicts not by individual enrolments but by *curricula*: pre-defined groups of courses that share students. Two courses in the same curriculum cannot be scheduled at the same time.

Central to ITC-2007, Track 3.

Definition 1.12.1 (Curriculum-Based Course Timetabling (CB-CTT, simplified)). Given:

- a set of courses, each with a number of weekly lectures, a teacher, and membership in one or more curricula;
- a set of rooms, each with a capacity;
- a set of time slots (periods $\times$ days);
- teacher unavailability constraints.

Hard constraints:

- H1. No two lectures of courses in the same curriculum at the same time.
- H2. No teacher teaches two courses at the same time.
- H3. Room capacity is respected.
- H4. Each course gets exactly its required number of lectures.

Soft constraints (penalised in the objective):

- S1. Room stability: a course should use the same room across all its lectures.
- S2. Minimum working days: lectures of a course should be spread over a minimum number of days.

S3. Curriculum compactness: lectures of the same curriculum on the same day should be in adjacent time slots (no “holes” in a student’s schedule).

High-school timetabling. Similar to course timetabling, but typically with tighter constraints: each class has a fixed weekly schedule, teachers must not have gaps between lessons, and there may be requirements on the distribution of subjects throughout the week (e.g. no more than two hours of maths per day).

1.12.2 Workforce timetabling: nurse rostering

Workforce timetabling assigns employees to shifts over a planning horizon, balancing coverage requirements with labour regulations and personal preferences. The most studied instance is the **nurse rostering problem** (NRP).

Definition 1.12.2 (Nurse Rostering Problem (simplified)). Given:

- a set of *nurses*, each with a *contract* (min/max working days, complete-weekend flag) and a set of *skills* (head nurse, regular nurse, trainee, ...);
- a set of *shift types* (Early, Late, Night, ...) with forbidden successions (e.g. Night $\rightarrow$ Early is not allowed);
- for each (shift, skill, day) triple, a *minimum* and *optimal* staffing requirement;
- a set of nurse *shift-off requests*: preferred days off or shifts to avoid.

A solution assigns each nurse, for each day of the planning horizon (typically one week), either a (shift, skill) pair or a day off.

Constraints. Hard constraints must always be satisfied:

- H1. *Single assignment*: at most one shift per nurse per day.
- H2. *Minimum staffing*: each (shift, skill, day) must meet the minimum requirement.
- H3. *Shift successions*: no forbidden day-to-day shift sequences.
- H4. *Skill matching*: a nurse can only cover a skill she possesses.

Soft constraints contribute to the objective function (lower is better), each weighted:

- S1. *Optimal coverage* (weight 3): each nurse below the optimal requirement is penalised.
- S2. *Preferences* (weight 1): each assignment to an undesired shift is penalised.
- S3. *Complete weekend* (weight 3): if flagged, the nurse should work both Sat and Sun or neither.
- S4. *Total assignments* (weight 2): working days outside the contract’s [min, max] range are penalised.

Example 1.12.3 (NR-05: a five-nurse instance). The smallest benchmark instance has 5 nurses, 2 skills, 3 shift types, and a one-week horizon. The instance file follows a structured text format:

```
SKILLS = 2
HeadNurse
Nurse
```

```

SHIFT_TYPES = 3
Early 0
Late 1 Early
Night 2 Early Late

CONTRACTS = 2
FullTime (4,5) 1
PartTime (2,3) 1

NURSES = 5
Patrick FullTime 2 HeadNurse Nurse
Andrea FullTime 2 HeadNurse Nurse
Stefaan PartTime 2 HeadNurse Nurse
Sara PartTime 1 Nurse
Nguyen FullTime 1 Nurse

```

Listing 1.2: Excerpt from the NR-05 instance file.

A solution is a list of (nurse, day, shift, skill) assignments. Benchmark instances with 12, 21, 30, 40, 50, 60, and 80 nurses are available in the course materials.

Derived from INRC-II.

1.12.3 Sport timetabling: round-robin tournaments

In a *round-robin tournament*, every team plays every other team exactly once (single round-robin) or twice (double round-robin, with home and away legs). The scheduling problem: assign matches to *rounds* (time slots) so that each team plays exactly once per round, while respecting venue constraints (home/away balance, travel minimisation, broadcaster requests, etc.).

Canonical 1-factorisation. For n teams (with n even), a single round-robin requires $n - 1$ rounds of $n/2$ simultaneous matches. A classical construction—the *canonical pattern*—fixes team n and rotates the remaining $n - 1$ teams around a circle:

Definition 1.12.4 (Canonical round-robin schedule). For n teams (n even), in round i ($i = 1, \dots, n - 1$):

- Team n plays team i (home/away alternates by parity of i).
- For $k = 1, \dots, n/2 - 1$: team $((k + i - 1) \bmod (n - 1)) + 1$ plays team $((i - k - 1) \bmod (n - 1)) + 1$, with home/away determined by parity of k .

This gives a valid schedule instantly (no search needed), but it does not optimise for travel distance or home/away patterns. Real-world sport scheduling adds many constraints, making it a rich combinatorial problem.

Example 1.12.5 (Round-robin for 6 teams). Applying the canonical pattern with $n = 6$:

Round	Matches
1	1–6, 3–5, 4–2
2	6–2, 4–1, 5–3
3	3–6, 5–2, 1–4
4	6–4, 1–3, 2–5
5	5–6, 2–4, 3–1

Figure 1.11: Canonical round-robin schedule for 6 teams. In each match “ a – b ”, team a is home. Every team appears exactly once per round.

```

1 #include <iostream>
2 using namespace std;
3
4 int Mod(int j, int n) {
5     if (j < 0) j += n;
6     j = j % n;
7     return (j == 0) ? n : j;
8 }
9
10 int main(int argc, char* argv[]) {
11     if (argc != 2) {
12         cerr << "Usage: " << argv[0]
13             << " <number of teams (even)>" << endl;
14         return 1;
15     }
16     int n = atoi(argv[1]);
17     if (n % 2 != 0) {
18         cerr << "Number of teams must be even" << endl;
19         return 1;
20     }
21     for (int i = 1; i < n; i++) {
22         cout << "Round " << i << ": ";
23         if (i % 2 == 1)
24             cout << i << '–' << n << " ";
25         else
26             cout << n << '–' << i << " ";
27         for (int k = 1; k < n / 2; k++)
28             if (k % 2 == 1)
29                 cout << Mod(k+i, n-1) << '–'
30                     << Mod(i-k, n-1) << " ";
31             else
32                 cout << Mod(i-k, n-1) << '–'
33                     << Mod(k+i, n-1) << " ";
34         cout << endl;
35     }
36     return 0;
37 }

```

Listing 1.3: Canonical round-robin pattern generator (compile with `g++ -Wall -o pattern pattern.cpp`).

■ Intermezzo — Timetabling as extended graph colouring

All three families of timetabling—educational, workforce, and sport—can be viewed as generalisations of graph colouring. In examination timetabling the connection is direct: exams are

vertices, conflicts are edges, time slots are colours. In nurse rostering, the “colours” become (shift, skill) pairs and the constraints are far richer, but the core combinatorial structure is the same: assign labels to entities so that incompatible assignments are avoided. This is why graph colouring heuristics like DSatur often serve as building blocks inside more sophisticated timetabling solvers.

1.13 Routing Problems

The last family of problems in our survey comes from **logistics**: given a set of locations to visit and distances between them, find the cheapest way to serve everyone.

TSP and VRP are among the most studied problems in combinatorial optimisation.

1.13.1 The Travelling Salesman Problem (TSP)

Definition 1.13.1 (Travelling Salesman Problem). Given a complete, undirected, edge-weighted graph $G = (V, E, c)$ with n nodes and edge costs $c_{ij} \geq 0$, find a *Hamiltonian cycle* (a cycle that visits every node exactly once) of minimum total cost.

In plain words: a salesperson must visit every city and return home, travelling the shortest possible route.

The number of distinct Hamiltonian cycles on n nodes is $(n - 1)!/2$, which grows faster than any exponential. TSP is NP-hard; there is no known polynomial-time exact algorithm.

A non-complete graph can be made complete by adding shortest-path edges.

Greedy 1: Nearest Neighbour. Start at a *depot* node. At each step, move to the *closest unvisited* node. After all nodes are visited, return to the depot.

Example 1.13.2 (Nearest Neighbour). On a small graph with 8 nodes, suppose the nearest-neighbour tour starting from node 1 follows $1 \rightarrow 2 \rightarrow 3 \rightarrow 8 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 1$ with total cost 24. The algorithm is fast— $\mathcal{O}(n^2)$ —but can make globally poor choices: a city left until last may force a long detour back.

Remark 1.13.3. Any tour with *crossing edges* is provably suboptimal: the crossing can always be “uncrossed” to reduce the cost. Nearest Neighbour frequently produces crossings, which is a visual sign that the solution can be improved.

Greedy 2: Insertion. Start with a small subtour (e.g. the depot plus one node). At each step, choose an unvisited node and a position in the current tour, and *insert* the node where it increases the tour cost the least. Variants differ in how the next node is chosen:

- *Nearest insertion*: pick the unvisited node closest to the current tour.
- *Farthest insertion*: pick the farthest—this often outperforms nearest insertion because it builds the “skeleton” of the tour early.
- *Cheapest insertion*: pick the (node, position) pair that causes the smallest cost increase.

Insertion heuristics run in $\mathcal{O}(n^2)$ to $\mathcal{O}(n^3)$ depending on the variant, and tend to produce fewer crossings than Nearest Neighbour.

1.13.2 Vehicle Routing Problem (VRP)

Definition 1.13.4 (Capacitated Vehicle Routing Problem—CVRP). Given:

- a complete, edge-weighted graph with a distinguished *depot* node;
- n *customers*, each with a demand $q_i > 0$;
- a fleet of K identical vehicles, each with capacity Q .

Find a set of K routes (cycles starting and ending at the depot) such that:

1. every customer is visited by exactly one route;
2. the total demand on each route does not exceed Q ;
3. the total travel cost is minimised.

The CVRP generalises the TSP: with $K = 1$ and $Q = \infty$ it reduces to a TSP. The extra dimension—splitting customers across vehicles while respecting capacity—makes the problem significantly harder.

Greedy approaches for VRP. All VRP heuristics come in *sequential* (build one route at a time) and *parallel* (build all routes simultaneously) flavours.

Nearest Neighbour (sequential). Build one route: start at the depot, repeatedly visit the nearest unvisited customer whose demand fits the remaining capacity. When no more customers fit, return to the depot and start a new route. Risk: the last vehicle inherits far-flung customers.

Nearest Neighbour (parallel). Start all vehicles at the depot. At each step, extend the vehicle whose nearest reachable customer is closest. This avoids the “last vehicle gets the leftovers” problem.

Clarke & Wright Savings.

1. Start with n trivial routes: depot $\rightarrow$ customer $i \rightarrow$ depot, one per customer.
2. For every pair of routes, compute the *saving* $s_{ij} = c_{0i} + c_{0j} - c_{ij}$: the cost saved by merging the two routes instead of making two separate round-trips.
3. Greedily merge the pair with the largest saving, provided the merged route respects capacity. Repeat until no more feasible merges improve cost.

Clustering + TSP.

1. Partition customers into K clusters (e.g. by geographic proximity), respecting vehicle capacity.
2. Solve a separate TSP within each cluster.

A variant: instead of explicit clustering, identify K *pivot* nodes (the farthest-apart customers), assign each vehicle to a pivot, and run Nearest Neighbour from there.

1.13.3 VRP variants

The CVRP is the simplest member of a large family. Common extensions include:

- **Time windows (CVRPTW).** Each customer has a time interval $[e_i, l_i]$: the vehicle may arrive early (and wait), but not late. Waiting time becomes part of the route cost.
- **Heterogeneous fleet.** Vehicles differ in capacity, cost per km, or customer compatibility (e.g. some trucks cannot reach certain locations).
- **Multi-depot.** Several depots; each vehicle departs from (and may return to) any depot.
- **Prizes / Orienteering.** Not all customers must be visited. Each customer offers a *prize*; the objective balances travel cost against collected prizes. With one vehicle this is the *orienteering problem*; with several, the *team orienteering problem*.
- **Pickup and delivery.** Some customers supply goods, others receive them. Capacity must be checked at every point along the route, making the problem significantly harder.
- **Time-dependent costs.** Travel times depend on the time of day (rush hour, traffic). Routes become asymmetric: the cost of an edge depends on when it is traversed.
- **Electric vehicles.** Limited range; the vehicle must visit recharge stations. Route planning must account for battery state.
- **Vehicles and drones.** A truck carries a drone that can make short deliveries from the truck's current position, saving the truck from detours.
- **Refrigerated vehicles.** Energy cost for refrigeration depends on ambient temperature, which varies along the route and over time.

1.14 Bin Packing

Bin packing generalises the knapsack: instead of one container, we have as many as we need, and the goal is to use *as few as possible*.

Knapsack: what fits in one bin?
Bin packing: how many bins for everything?

1.14.1 1D Bin Packing

Definition 1.14.1 (1D Bin Packing). Given n items with sizes $w_1, \dots, w_n$ and bins of capacity W , assign every item to a bin so that the total size in each bin does not exceed W and the number of bins used is minimised.

Example 1.14.2 (Six items, capacity 50). Items: 30, 4, 7, 12, 6, 40. Capacity $W = 50$.

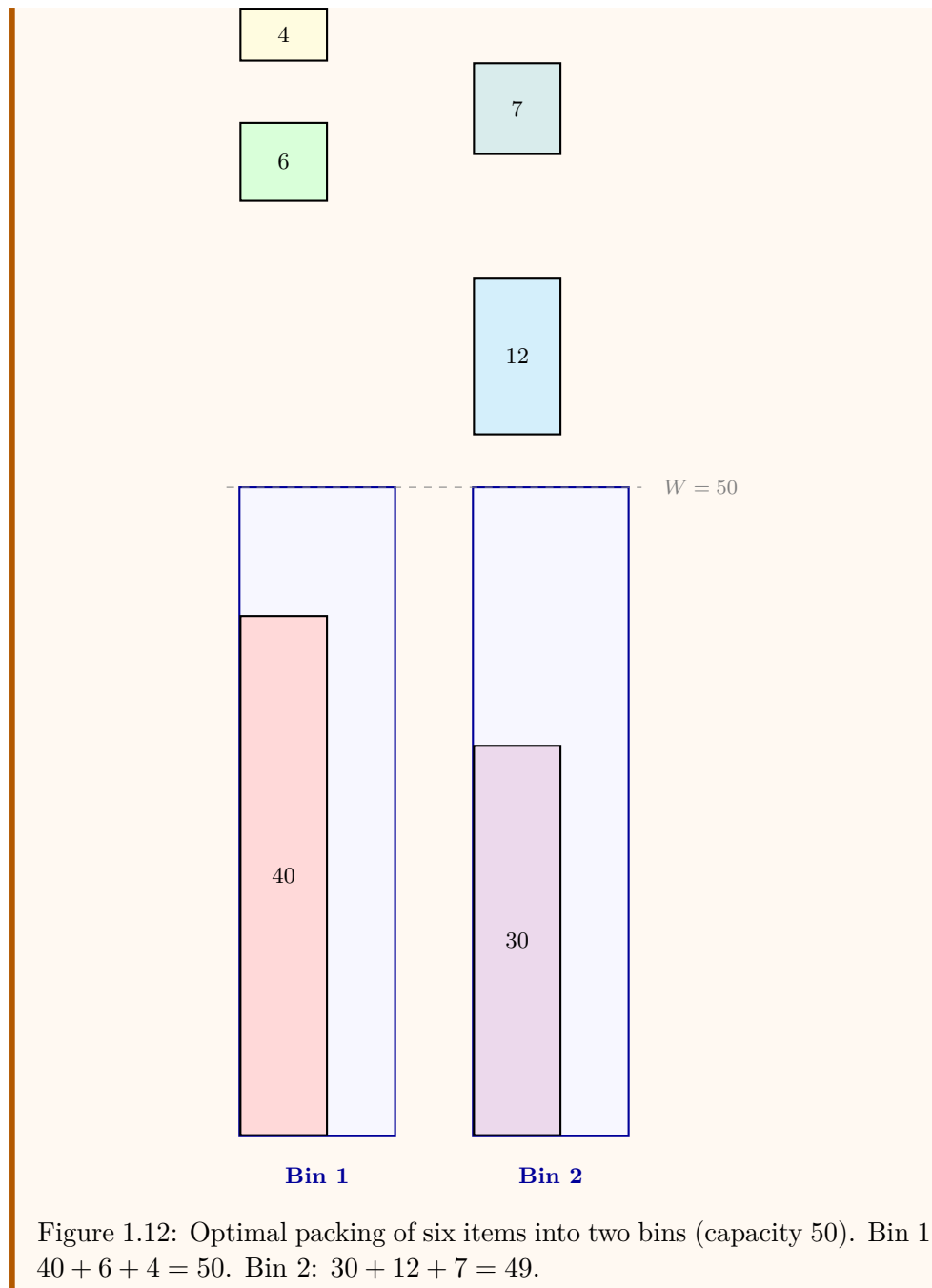


Figure 1.12: Optimal packing of six items into two bins (capacity 50). Bin 1: $40 + 6 + 4 = 50$. Bin 2: $30 + 12 + 7 = 49$.

Greedy heuristics for 1D Bin Packing.

First Fit (FF). Process items in the given order. Place each item in the *first* bin where it fits; if none, open a new bin.

Best Fit (BF). Process items in the given order. Place each item in the bin that will have the *least remaining capacity* after insertion (tightest fit); if none, open a new bin.

First Fit Decreasing (FFD) / Best Fit Decreasing (BFD). Sort items by *decreasing* size, then apply FF or BF. Sorting large items first is a simple but effective improvement: large items are the hardest to fit, so placing them early avoids situations where a large item forces an extra bin at the end.

Example 1.14.3 (Comparing heuristics on theorem 1.14.2). With items in the original order 30, 4, 7, 12, 6, 40:

- **First Fit:** Bin 1 = {30, 4, 7, 6} (47), Bin 2 = {12} (12), Bin 3 = {40} (40). **3 bins.**
- **Best Fit:** Bin 1 = {40, 7} (47), Bin 2 = {30, 12, 6} (48), Bin 3 = {4} (4). **3 bins.**

With items sorted decreasingly (40, 30, 12, 7, 6, 4):

- **Best Fit Decreasing:** Bin 1 = {40, 7} (47), Bin 2 = {30, 12, 6, ?}—12 + 30 = 42, then 6 fits (48), then 4 fits *only in Bin 1* (47 + 4 > 50, so new bin). Actually: Bin 1 = {40, 7}, Bin 2 = {30, 12, 6}, Bin 3 = {4}. Still **3 bins**—but on many instances BFD significantly outperforms FF.

The optimum (2 bins) requires a more global view: Bin 1 = {40, 6, 4} (50), Bin 2 = {30, 12, 7} (49).

1.14.2 2D and 3D Bin Packing

In higher dimensions, items and bins become geometric objects:

2D Bin Packing. Items are *rectangles*; bins are larger rectangles. Beyond deciding *which* bin, we must decide *where* in the bin to place each item (coordinates) and possibly whether to *rotate* it 90°.

Placement heuristics typically maintain a list of “corner points”—positions where a new item can be placed flush against existing items or bin walls—and choose the corner that wastes the least space.

- **Strip packing:** pack items into horizontal strips; each strip is as tall as its tallest item. Simple but wastes vertical space.
- **Shelf algorithms:** a refinement of strip packing with rules for starting new shelves.

Applications: cutting glass, wood, or sheet metal; VLSI layout; page layout.

Remark 1.14.4 (Guillotine cuts). In glass and wood cutting, every cut must go *edge to edge* (a “guillotine cut”). This rules out L-shaped residual pieces: after cutting, both resulting pieces must be rectangles. The constraint significantly restricts the set of feasible packings and requires specialised algorithms.

3D Bin Packing / Container Loading. Items are *boxes* (rectangular parallelepipeds); bins are shipping containers. Additional real-world constraints include:

- **Stability:** items must rest on a flat surface; no floating items.
- **Fragility:** heavy items cannot be stacked on fragile ones.
- **Weight balance:** the centre of gravity must stay within tolerance (especially for trucks and aircraft).
- **Rotation restrictions:** some items may only rest on certain faces (e.g. “this side up”).
- **Loading order:** items needed first at delivery must be accessible (loaded last, near the door).

■ Intermezzo — From theory to practice

The gap between textbook bin packing and industrial applications is large. A real glass-cutting optimiser must handle guillotine constraints, material defects, different glass types, and client grouping (pieces for the same order should come from the same sheet to simplify logistics). A container-loading solver must model gravity, fragility, and access order. The greedy heuristics we have seen—First Fit, Best Fit, and their decreasing variants—serve as building blocks inside these richer systems, often as the constructive phase of a metaheuristic.

Chapter 2

C++ Implementation Framework

2.1 C++ Recap

From this point on we *implement* the algorithms we study. The language is C++ (compiled with `g++` or `clang++`). This section recaps the OOP features we rely on; students comfortable with all of them can skip ahead to section [2.2](#).

2.1.1 Encapsulation and information hiding

A **class** bundles *data members* (the state) and *methods* (the operations on that state) into a single unit. Access control enforces *information hiding*:

- **private** (default for **class**): accessible only within the class itself.
- **protected**: accessible within the class and its subclasses.
- **public**: accessible from anywhere.

```
1 class Graph {
2 public:                                // interface
3     Graph(string file_name);          // constructor
4     unsigned Nodes() const { return nodes; }
5     bool Arc(unsigned i, unsigned j) const;
6 protected:                           // internal state
7     unsigned nodes;
8     vector<vector<bool>> adjacency_matrix;
9 };
```

Listing 2.1: Basic class layout.

The `const` qualifier on `Nodes()` promises that calling it does not modify the object—a crucial contract for readers of the code and for the compiler’s optimiser.

2.1.2 Constructors

A **constructor** initialises an object when it is created. It has the same name as the class and no return type. For our optimisation code, the constructor typically reads an instance from a file:

```
1 Graph::Graph(string file_name) {
2     ifstream is(file_name);
3     if (!is) {
4         cerr << "Could not open " << file_name << endl;
```

```

5     exit(1);
6 }
7 // ... parse the file and fill data members ...
8 }

```

Listing 2.2: Constructor that reads from a file.

2.1.3 File I/O: ifstream and ofstream

The standard library provides `ifstream` (input) and `ofstream` (output) in `<fstream>`. They behave like `cin/cout` but read from / write to files:

```

1 #include <fstream>
2
3 ifstream is("instance.txt"); // open for reading
4 unsigned n;
5 is >> n;                      // read an integer
6
7 ofstream os("solution.txt"); // open for writing
8 os << "makespan = " << cmax << endl;

```

Listing 2.3: Reading and writing files.

2.1.4 The vector container

`std::vector<T>` is a dynamically-sized array. We use it everywhere instead of raw C arrays:

```

1 vector<int> v(10, 0); // 10 elements, all zero
2 v.push_back(42);     // append
3 v.size();            // current length
4 v[3] = 7;            // random access (no bounds
   check)
5
6 // 2D: vector of vectors
7 vector<vector<bool>> matrix(n, vector<bool>(n, false));

```

Listing 2.4: Common vector operations.

For adjacency matrices we use `vector<vector<bool>>` (compact, random access); for adjacency lists, `vector<vector<unsigned>>` (compact, iteration-friendly).

2.2 Software Architecture for Optimisation

Every solver we build in this course follows the same three-module architecture:

Separating I/O from the solver lets us swap algorithms without touching file-parsing code.

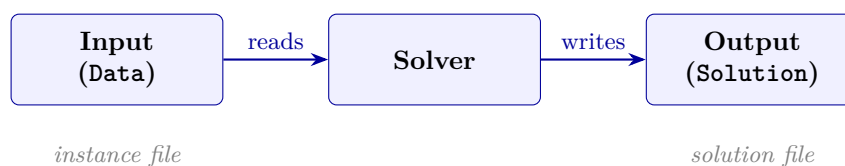


Figure 2.1: Three-module architecture. The **Input** class reads the instance file and stores the problem data. The **Solver** reads from Input and writes into a **Solution** object. The Solution class handles output formatting.

1. **Input / Data class.** Reads and stores the instance. Provides read-only accessors (`const` methods) so that the solver cannot accidentally modify the input.
2. **Output / Solution class.** Stores the current solution and provides operators for modifying and printing it.
3. **Solver.** Contains the algorithm logic. It receives a `const` reference to the Input and produces a Solution. Swapping solvers (greedy, backtracking, local search) requires no changes to the I/O classes.

This separation is not just good software engineering—it is essential when comparing multiple algorithms on the same instances, which is exactly what the course project requires.

2.3 Example: Graph Colouring I/O Classes

We illustrate the architecture on the graph colouring problem. The two files `GC_Data.hh` (header) and `GC_Data.cc` (implementation) define the Input and Output classes.

2.3.1 The Graph class (Input)

```

1 class Graph {
2     friend ostream& operator<<(ostream& os, const Graph& g);
3 public:
4     Graph(string file_name);
5     unsigned Nodes() const { return nodes; }
6     unsigned Colors() const { return colors; }
7     bool Arc(unsigned i, unsigned j) const
8     { return adjacency_matrix[i][j]; }
9     unsigned Degree(unsigned i) const
10    { return adjacency_list[i].size(); }
11    unsigned ArcListMember(unsigned i, unsigned j) const
12    { return adjacency_list[i][j]; }
13 protected:
14    unsigned nodes, colors;
15    vector<vector<bool>> adjacency_matrix;
16    vector<vector<unsigned>> adjacency_list;
17 };

```

Listing 2.5: `GC_Data.hh` — the Graph class (input data).

Design notes.

- The graph is stored in *two* representations: an adjacency matrix (fast $\mathcal{O}(1)$ edge queries) and adjacency lists (fast neighbour iteration). Both are built once by the constructor.
- All accessors are `const`—the solver can read but never modify the graph.
- The `friend` declaration lets the `<<` operator access private members for printing.
- Data members are `protected` (not `private`) so that subclasses—e.g. a `WeightedGraph`—can extend them.

The constructor reads a MiniZinc-style data file:

```

1 Graph::Graph(string file_name) {
2     char ch; string buffer; unsigned i, j, val;
3     ifstream is(file_name);
4     if (!is) { cerr << "Could not open " << file_name <<
5         endl; exit(1); }
6
7     is >> buffer >> ch >> nodes >> ch; // "nodes = <n>";
8     is >> buffer >> ch >> colors >> ch; // "colors = <k>";
9
10    adjacency_matrix.resize(nodes, vector<bool>(nodes, false));
11    adjacency_list.resize(nodes);
12
13    is >> buffer >> ch >> ch >> ch; // "
14    adjacency_matrix = [/"
15    for (i = 0; i < nodes; i++)
16        for (j = 0; j < nodes; j++) {
17            is >> val >> ch;
18            if (val == 1 && i < j) { // upper triangle
19                only
20                adjacency_matrix[i][j] = true;
21                adjacency_matrix[j][i] = true;
22                adjacency_list[i].push_back(j);
23                adjacency_list[j].push_back(i);
24            }
25        }
26    }
27 }
```

Listing 2.6: Constructor: reading the instance file.

The file format matches MiniZinc’s data syntax (which we will use in the next lectures), so the same instance file serves both the C++ solver and the MiniZinc model.

2.3.2 The Coloring class (Output)

```

1 class Coloring {
2     friend ostream& operator<<(ostream& os, const Coloring&
3         col);
4 public:
5     Coloring(const Graph& my_in)
6         : in(my_in), coloring(in.Nodes(), -1) {}
7     void Reset();
8     int operator[](unsigned i) const { return coloring[i]; }
9     int& operator[](unsigned i) { return coloring[i]; }
10 protected:
11     const Graph& in;
12     vector<int> coloring;
13 };
```

Listing 2.7: GC_Data.hh — the Coloring class (solution).

Design notes.

- The constructor takes a `const Graph&` and initialises every node's colour to -1 (uncoloured).
- The `operator[]` overloads provide array-like syntax: `col[v] = 3` assigns colour 3 to node v ; `col[v]` reads it. The `const` version returns by value (read-only); the non-`const` version returns by reference (read-write).
- `Reset()` sets all colours back to -1 , allowing the solver to restart without reallocating memory.
- The `in` reference lets the output class access graph data (e.g. number of nodes) without duplicating it.

Observation 2.3.1. This pattern—Data class with `const` accessors, Solution class with a reference to the data—recurs in every problem we implement. For TSP: `TSP_Data` stores the distance matrix, `TSP_Solution` stores the tour. For bin packing: `BP_Data` stores item sizes and bin capacity, `BP_Solution` stores the assignment of items to bins. The solver is always a separate function or class that reads from Data and writes to Solution.

2.4 Implementing Greedy: DSatur for Graph Colouring

With the architecture in place, let us implement the DSatur algorithm from section 1.11.2. The solver lives in its own compilation unit (`GC_Greedy.hh/cc`), separate from the I/O classes.

2.4.1 The solver loop

```

1 void GreedyGCSolver(const Graph& in, Coloring& out) {
2     unsigned i, k, c;
3     int n;
4     // colors[v] = set of colours still available for vertex
5     // v
6     vector<vector<unsigned>> colors(
7         in.Nodes(), vector<unsigned>(in.Colors()));
8     for (i = 0; i < in.Nodes(); i++)
9         iota(colors[i].begin(), colors[i].end(), 0);
10
11    out.Reset();
12    for (i = 0; i < in.Nodes(); i++) {
13        n = SelectNode(colors, in, out); // DSatur selection
14        if (colors[n].size() == 0)
15            return; // no feasible colour: search fails
16        k = Random(0, colors[n].size() - 1);
17        c = colors[n][k]; // random among
18        // available
19        Update(n, c, colors, in); // propagate
20        out[n] = c;
21    }
22 }
```

Listing 2.8: DSatur main loop (`GC_Greedy.cc`).

Key data structure. `colors[v]` is the list of colours still available for vertex v . Initially every vertex has all k colours. After colouring a vertex,

`Update` removes that colour from every neighbour's list. The *saturation degree* of v is implicitly $k - |\text{colors}[v]|$: fewer available colours = higher saturation.

2.4.2 Node selection (DSatur rule)

```

1 unsigned SelectNode(const vector<vector<unsigned>>& v,
2                     const Graph& in, const Coloring& out)
3 {
4     unsigned i;
5     int best = -1;
6     for (i = 0; i < v.size(); i++)
7         if (out[i] == -1 &&                                // uncoloured
8             (best == -1
9              || v[i].size() < v[best].size()                // fewer
10               available
11               || (v[i].size() == v[best].size()
12                  && in.Degree(i) > in.Degree(best))))
13         best = i;                                          // tie-break:
14                                     degree
15 return best;
16 }
```

Listing 2.9: `SelectNode`: pick the most saturated uncoloured vertex.

Smaller `colors[v].size()` means higher saturation. Ties broken by highest degree—exactly the DSatur rule.

2.4.3 Propagation

```

1 void Update(unsigned n, unsigned c,
2             vector<vector<unsigned>>& v, const Graph& in)
3 {
4     for (unsigned i = 0; i < in.Degree(n); i++) {
5         unsigned k = in.ArcListMember(n, i);
6         for (unsigned j = 0; j < v[k].size(); j++)
7             if (v[k][j] == c) {
8                 v[k].erase(v[k].begin() + j);
9                 break;
10            }
11 }
```

Listing 2.10: `Update`: remove colour c from all neighbours of n .

2.4.4 Driver and Makefile

```

1 #include "GC_Data.hh"
2 #include "GC_Greedy.hh"
3
4 int main(int argc, char* argv[]) {
5     if (argc != 2) {
6         cerr << "Usage: " << argv[0] << " <input_file>" <<
7             endl;
8         return 1;
9     }
10     Graph in(argv[1]);
```

```

10 Coloring out(in);
11 GreedyGCSolver(in, out);
12 cout << out << endl;
13 return 0;
14 }

```

Listing 2.11: Minimal driver (GC_Driver.cc).

The driver is trivially short: create Input, create Output, run Solver, print. To switch algorithms, only the solver call changes.

```

1 OPTIONS = -Wall -Wfatal-errors -O3
2
3 GC_Test.exe: GC_Driver.o GC_Greedy.o GC_Data.o Random.o
4     g++ -o GC_Test.exe $^
5
6 GC_Data.o: GC_Data.cc GC_Data.hh
7     g++ $(OPTIONS) -c GC_Data.cc
8
9 GC_Greedy.o: GC_Greedy.cc GC_Greedy.hh GC_Data.hh
10    g++ $(OPTIONS) -c GC_Greedy.cc
11
12 GC_Driver.o: GC_Driver.cc GC_Data.hh
13    g++ $(OPTIONS) -c GC_Driver.cc
14
15 clean:
16    rm -f *.o GC_Test.exe

```

Listing 2.12: Makefile for the graph colouring solver.

2.5 Example 2: Bus Driver Scheduling (I/O classes)

As a second example of the Input/Output pattern, we look at the **Bus Driver Scheduling** (BDS) problem: given a set of *tasks* (bus trips) and a set of *shifts* (driver work-blocks, each covering certain tasks), select a minimum-cost set of shifts that covers every task.

*BDS is a set-covering problem:
shifts = sets, tasks = elements.*

```

1 class BDS_Input {
2     friend ostream& operator<<(ostream& os, const BDS_Input&
3         bs);
4 public:
5     BDS_Input(string file_name);
6     unsigned Tasks() const { return tasks; }
7     unsigned Shifts() const { return shifts; }
8     bool Covers(unsigned s, unsigned t) const
9         { return covers[s][t]; }
10    unsigned ShiftCost(unsigned s) const
11        { return shift_cost[s]; }
12 protected:
13    unsigned tasks, shifts;
14    vector<unsigned> shift_cost;
15    vector<vector<bool>> covers;
16    vector<vector<unsigned>> shift_members;
17    vector<vector<unsigned>> task_coverage;
18 };

```

Listing 2.13: BDS_Data.hh — Input class (excerpt).

```

1 class BDS_Output {
2     friend ostream& operator<<(ostream& os, const BDS_Output
      & out);
3 public:
4     BDS_Output(const BDS_Input& i);
5     bool operator[](unsigned s) const
6     { return selected_shifts[s]; }
7     unsigned Uncovered() const { return uncovered; }
8     unsigned TotalCost() const { return total_cost; }
9
10    void InsertShift(unsigned s);
11    void RemoveShift(unsigned s);
12    void RemoveAll();
13 private:
14    const BDS_Input& in;
15    vector<bool> selected_shifts;
16    vector<unsigned> coverage;
17    unsigned uncovered, total_cost;
18 };

```

Listing 2.14: BDS_Data.hh — Output class (excerpt).

Design notes.

- The Output class maintains *incremental* bookkeeping: `InsertShift` and `RemoveShift` update `coverage`, `uncovered`, and `total_cost` without a full recount.
- This incremental pattern will be essential for *local search* (later chapters), where we repeatedly insert/remove shifts and need fast cost evaluation.
- The same Input/Output split applies: greedy, backtracking, or local-search solvers all use the same `BDS_Input` and `BDS_Output`.

Chapter 3

Single-Machine Scheduling

Chapter 4

Parallel-Machine Scheduling

Chapter 5

Flow Shop & Job Shop Scheduling

Chapter 6

Metaheuristics for Scheduling